# Sovereign Acceptance Runtime: Verifiable Human Ratification and Evidentiary Audit Trails for Autonomous AI Agents under the EU AI Act

**Working Paper v1.1** (revises v1.0 of August 2026; see "Revision Notes" at the end)

**Date:** October 2026

**Author:** Alen Mikulić, Independent Researcher, Zagreb, Croatia

**Status:** Preprint · DOI: 10.5281/zenodo.23270905

## Abstract

Autonomous AI agents are rapidly evolving from conversational assistants into software systems capable of invoking external services, modifying persistent state, and executing transactions with real-world consequences. While contemporary agent frameworks frequently incorporate human-in-the-loop (HITL) approval workflows, these mechanisms typically provide operational control without generating portable, cryptographically verifiable evidence of who authorized an action, which execution payload was reviewed, and whether the executed action remained identical to the approved one.

The regulatory significance of this limitation has increased with Regulation (EU) 2024/1689 (the EU AI Act) [1]. Under Article 113 as amended by Regulation (EU) 2026/1744 (the Digital Omnibus on AI) [2], high-risk AI systems become subject to mandatory requirements for automatic event logging, technically implemented human oversight, and operational log retention from 2 December 2027 (systems listed in Annex III) and 2 August 2028 (systems covered by Annex I Union harmonisation legislation). Although these obligations establish regulatory objectives, neither the Regulation nor currently available technical standards specify how authorization records should acquire independent evidentiary value or cryptographic resistance to undetected modification. This creates an architectural gap between regulatory requirements and implementation guidance.

This paper introduces the Sovereign Acceptance Runtime (SAR), an execution architecture that models human authorization as a cryptographically verifiable ratification bound to the exact execution payload. Concurrent IETF Internet-Drafts already define pre-execution authorization records and human-signed action approvals; SAR's contribution is their composition into a single profile. Building upon the IETF Supply Chain Integrity, Transparency, and Trust (SCITT) architecture (RFC 9943) [3] and CBOR Object Signing and Encryption (COSE) signed statements, SAR represents authorization as a machine-enforceable Registration Policy evaluated before execution. The runtime adopts a fail-closed execution model in which actions proceed only when accompanied by verifiable authorization evidence whose payload binding remains valid at the moment of execution.

The paper develops a threat model and six architectural requirements for verifiable human ratification, describes a reference architecture implementing these requirements, positions the work against concurrent IETF drafts on agent authorization, relates the proposed mechanisms to selected provisions of the EU AI Act, evaluates a reference implementation using COSE_Sign1 and deterministic CBOR against an implementation-independent conformance suite whose adequacy is checked by mutation analysis, and outlines a potential path toward interoperability within the IETF SCITT ecosystem.

## 1. Introduction

Artificial intelligence systems are undergoing a transition from passive conversational interfaces toward autonomous software components capable of acting directly upon external environments. Modern AI agents no longer operate exclusively by generating information for human interpretation; they increasingly invoke application programming interfaces (APIs), modify persistent data stores, orchestrate infrastructure, execute administrative workflows, and initiate financial or operational transactions.

This evolution fundamentally changes the security and governance problem surrounding AI deployment. When autonomous systems become capable of producing irreversible external effects, human authorization can no longer be treated merely as a user-interface interaction. Instead, authorization itself becomes a security artifact whose authenticity, integrity, and evidentiary value determine whether an autonomous action should be permitted to occur.

Most existing human-in-the-loop (HITL) mechanisms successfully provide operational oversight by pausing execution until a human approves or rejects an intended action. However, approval is commonly represented as transient application state rather than as independently verifiable cryptographic evidence. Once execution resumes, conventional systems typically retain only mutable audit logs or workflow records describing that an approval occurred. Such records generally do not provide transferable evidence demonstrating who approved the action, what exact payload was reviewed, whether the executed payload remained identical to the approved payload, or whether the authorization can later be verified independently of the original runtime.

This limitation has become increasingly relevant in regulated environments. Regulation (EU) 2024/1689 [1] establishes mandatory obligations for high-risk AI systems. As amended by Regulation (EU) 2026/1744 [2], these obligations apply from 2 December 2027 for systems listed in Annex III and from 2 August 2028 for systems covered by Annex I; the transparency obligations of Article 50 apply from 2 August 2026. Among these obligations are automatic recording of operational events, technically implemented human oversight, and minimum retention periods for operational records. These provisions define regulatory objectives while remaining largely technology-neutral. They do not prescribe specific cryptographic mechanisms through which authorization events become independently verifiable or acquire evidentiary value suitable for regulatory review, forensic investigation, or third-party audit. At the time of writing, harmonized technical standards addressing AI logging and human oversight remain under development (the amending Regulation itself cites the delayed availability of standards as a reason for the deferral [2, Recital 40]), leaving implementers without a standardized architecture for producing verifiable authorization evidence.

This paper proposes the Sovereign Acceptance Runtime (SAR) to address that architectural gap. Rather than treating authorization as an ephemeral workflow event, SAR models human ratification as a signed cryptographic statement evaluated before execution. Authorization becomes a first-class protocol object bound to the exact execution payload and validated through a machine-enforceable Registration Policy before any protected action is permitted to proceed. Successful authorization produces portable cryptographic evidence that remains independently verifiable after execution, while unsuccessful verification results in deterministic denial of execution according to a fail-closed execution model.

The architecture intentionally builds upon existing Internet standards rather than introducing proprietary cryptographic primitives. Human ratification is represented using COSE signed statements, transparency evidence is provided through SCITT-compatible transparency services, and authorization semantics are expressed through Registration Policies evaluated before ledger registration. This separation allows the proposed architecture to reuse established cryptographic building blocks while introducing new semantics governing autonomous execution.

### 1.1 Research Objective

The objective of this work is to investigate whether existing transparency and signing technologies can be combined into an execution architecture in which human authorization becomes independently verifiable, cryptographically bound to the executed payload, and portable across organizational trust boundaries.

Rather than proposing a new digital signature algorithm or transparency mechanism, the paper investigates how existing standards may be composed to produce authorization evidence suitable for autonomous AI systems operating in regulated environments.

### 1.2 Research Contributions

This paper makes four principal contributions.

- **C1. Threat and Requirements Model.** It develops an adversarial threat model for autonomous AI execution and derives six architectural requirements governing verifiable human ratification.
- **C2. Sovereign Acceptance Runtime.** It proposes an execution architecture that combines SCITT transparency services, COSE signed statements, and policy-governed authorization into a deterministic fail-closed execution pipeline. Individual elements of this pipeline appear in concurrent Internet-Drafts (Section 2.6); the contribution is their composition, in particular the use of the SCITT Registration Policy as the pre-execution authorization gate for human ratifications.
- **C3. Reference Implementation and Conformance Suite.** It provides a reference implementation of the core authorization path and an implementation-independent conformance suite (55 tests, including differential testing against a model of the specification), whose adequacy is checked by mutation analysis. Transparency Service integration is not yet implemented (Section 6).
- **C4. Standards Pathway.** It outlines a potential interoperability path positioning SAR as an application profile built upon existing IETF technologies rather than as a standalone authorization protocol.

### 1.3 Truth-State Convention

To distinguish architectural intent from implementation maturity, technical claims throughout this paper are explicitly classified according to one of three implementation states.

- **[Specified]** — fully defined within the architectural model but not completely implemented.
- **[Partially Enforced]** — implemented within software components while requiring broader operational validation.
- **[Proven]** — demonstrated within the reference implementation through automated verification of the claimed behavior.

These classifications are applied consistently throughout the architectural and implementation sections of the paper.

### 1.4 Scope

The Sovereign Acceptance Runtime represents one possible technical architecture for producing verifiable authorization evidence for autonomous AI systems. It is not presented as the only implementation capable of supporting the evidentiary objectives discussed, nor as a normative interpretation of Regulation (EU) 2024/1689. Questions of legal compliance remain subject to applicable legislation, future harmonized standards, deployment context, and competent supervisory authorities.

### 1.5 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews the regulatory background, relevant Internet standards, related systems, and concurrent work on agent authorization. Section 3 develops the threat model and derives the architectural requirements. Section 4 presents the Sovereign Acceptance Runtime architecture and authorization protocol. Section 5 relates the proposed mechanisms to selected provisions of the EU AI Act. Section 6 evaluates a reference implementation against a conformance suite. Section 7 discusses a possible standardization pathway. Section 8 examines limitations and future research directions. Section 9 concludes.

## 2. Background and Related Work

### 2.1 Regulatory Context

Regulation (EU) 2024/1689 [1] establishes a risk-based regulatory framework for artificial intelligence systems deployed within the European Union. For high-risk AI systems, the Regulation introduces legally binding obligations concerning lifecycle traceability, human oversight, technical documentation, and operational accountability.

Among the provisions most relevant to autonomous AI agents are the requirements for automatic recording of operational events (Article 12), technically implemented human oversight (Article 14), and retention of operational records by providers and deployers (Articles 19 and 26). These obligations (Chapter III, Sections 2 and 3) apply to high-risk AI systems from 2 December 2027 for Annex III systems and from 2 August 2028 for Annex I systems (Article 113, third paragraph, point (c), as amended by Regulation (EU) 2026/1744 [2]). The original Regulation had set 2 August 2026 for Annex III systems; the deferral was motivated in part by the delayed availability of harmonised standards [2, Recital 40]. The Regulation's penalty provisions (Chapter XII) have applied since 2 August 2025, with the exception of Article 101 [1, Art. 113].

Importantly, the Regulation specifies regulatory objectives rather than implementation mechanisms. It requires that operational events be recorded and that meaningful human oversight be technically supported, but it does not prescribe how authorization events should be represented cryptographically or how evidentiary integrity should be established.

At the time of writing, European harmonized standards addressing AI logging and human oversight remain under development. Draft efforts within CEN/CENELEC and ISO/IEC concerning AI system event recording and oversight provide important direction but do not yet define interoperable mechanisms for producing independently verifiable authorization evidence.

Consequently, implementers face a gap between regulatory obligations and technical realization. This paper addresses that gap from an architectural perspective.

### 2.2 Supply Chain Integrity, Transparency, and Trust (SCITT)

The Internet Engineering Task Force (IETF) Supply Chain Integrity, Transparency, and Trust (SCITT) Working Group develops interoperable mechanisms for publishing, registering, and verifying signed claims within transparency services. The SCITT architecture was published as a Standards Track RFC in June 2026 (RFC 9943 [3]); terminology in this paper (Signed Statement, Transparency Service, Receipt, Registration Policy) follows that document.

Rather than defining a particular transparency ledger implementation, SCITT specifies architectural components enabling independently verifiable provenance across organizational trust boundaries.

The architecture revolves around four principal concepts:

- **Signed Statements**, represented as cryptographically signed assertions describing an object or event.
- **Transparency Services**, responsible for evaluating submitted statements against Registration Policies before committing accepted statements to an append-only transparency log.
- **Receipts**, providing cryptographic evidence that a submitted statement has been accepted and registered.
- **Independent Verification**, allowing third parties to validate registered statements without trusting the original issuer.

SCITT deliberately remains application-neutral. It standardizes mechanisms for transparency and verification while leaving application-specific semantics—including authorization workflows—outside its scope.

The architecture proposed in this paper builds upon this extensibility rather than modifying the underlying SCITT model.

### 2.3 COSE and CBOR

SAR relies upon existing IETF cryptographic standards for representing signed authorization objects.

The Concise Binary Object Representation (CBOR) (RFC 8949 [4]) provides a compact binary encoding suitable for structured protocol objects; its deterministic encoding rules (Section 4.2) are relied upon in Section 4.4.

CBOR Object Signing and Encryption (COSE) (RFC 9052 [5], RFC 9053 [6]) defines interoperable formats for digital signatures, message authentication, and encryption over CBOR objects. RFC 9864 [7] deprecates polymorphic algorithm identifiers such as EdDSA in favour of fully specified ones; a SAR profile using Ed25519 should therefore use the fully specified Ed25519 identifier.

Within SAR, COSE signatures are used to bind both agent proposals and human ratifications to deterministic payload representations. Because SAR reuses existing standards rather than introducing new cryptographic formats, the proposed architecture remains compatible with established verification libraries and future interoperability efforts.

### 2.4 Human Oversight in Contemporary Agent Frameworks

Current AI agent frameworks increasingly expose mechanisms allowing users to review and approve actions before execution. These approval workflows typically provide:

- execution pausing,
- confirmation dialogs,
- user approval callbacks,
- policy gating,
- resumable execution.

Such mechanisms successfully coordinate operational workflows and improve user control. However, they generally treat authorization as transient runtime state. After execution resumes, approval commonly survives only as mutable application state or conventional audit records.

Consequently, existing workflows usually do not generate portable cryptographic artifacts capable of demonstrating:

- who approved an action;

- which exact payload was reviewed;
- whether execution matched the approved payload;
- whether authorization can later be independently verified.

Accordingly, current human-in-the-loop mechanisms primarily address operational governance rather than evidentiary authorization.

### 2.5 Related Systems

Several existing technologies address adjacent aspects of integrity, provenance, or governance.

- **Sigstore** [22] provides transparency-backed signatures for software artifacts, enabling independent verification of software origin and integrity.
- **in-toto** [21] records software supply-chain steps through signed attestations that verify whether software artifacts followed expected production workflows.
- **SCITT** [3] generalizes transparency services for signed claims across multiple application domains.
- **Remote attestation (RATS)** [20] lets a verifier appraise evidence about the state of a target environment. It attests to system state rather than to a human decision over a specific action.
- **Enterprise governance platforms** similarly provide approval workflows, policy engines, and trusted execution mechanisms supporting operational control.

These systems demonstrate mature approaches to software provenance, integrity, and governance. However, their primary objective differs from that considered in this paper. Rather than producing portable authorization evidence governing autonomous execution before an action occurs, they focus on software supply-chain integrity, infrastructure governance, or operational workflow management.

### 2.6 Concurrent Work on Agent Authorization Records

Since early 2026, several individual Internet-Drafts have addressed the evidentiary gap for AI agent actions directly. They are individual submissions, not IETF consensus documents, and are cited here as work in progress. Several of them predate the first version of this paper, and the elements they define overlap substantially with SAR.

- **Pre-execution permits bound to request bytes.** Munoz [8] (first posted May 2026) defines a SCITT profile for pre-execution AI action authorization records (“Permits”), committing a SHA-256 digest over the canonical wire-body bytes of the request to be dispatched. The issuer of a Permit is not specified to be a human. Enforcement is fail-closed only where dispatch transits a managed proxy, and registration APIs are out of scope. The draft states that no COSE_Sign1 envelope is emitted by the current implementation.
- **Human-signed approval of the exact action.** Schrock [9] (first posted June 2026) defines authorization receipts in which, before an irreversible action executes, an enrolled approver signs the exact action with an approver-held key; a denial is also signed. A companion draft [10] defines how named-human authorization evidence, including multi-approver quorum, is bound into agent-action records. Neither maps its mechanism to the EU AI Act or uses a SCITT Registration Policy as the gate.
- **Agent-to-human countersignature.** Kumaresan [11] (first posted July 2026) defines a protocol in which an agent emits an Intent and keyed approvers return signed approve or reject Countersignatures bound to it, with quorum and veto rules; a relying party must not act on an Intent that has not resolved to authorized. It does not use COSE or SCITT.
- **Post-hoc action records.** The Agent Action Capsule profile [12] records a capsule for every verdict, including refusals, as evidence that a gate worked, and defers the pre-execution permission question to permit profiles. Delegation receipts [13] bind a user’s signature to a scope of permitted agent

actions rather than to a single action. Further SCITT profiles define agent execution records mapped to AI Act Articles 12 and 19 [14], per-action receipts [15], and a human escalation mechanism [16]. A domain profile for the Operating Model Protocol maps agent decision logging to Article 12 and routes policy non-compliance to human oversight with reference to Article 14(4) [17].

- **Machine admission control.** He and Yu [18] describe certificate-bound admission for agentic infrastructure, a machine-level pre-execution check. Despite the similar name, it does not involve human ratification. Racioppi [19] enforces digitally signed standing mandates on AI agent tool calls with chained audit trails and an Article 12/14 mapping.

### 2.7 Architectural Gap

Taken together, existing standards and concurrent drafts establish the individual building blocks: signed statements and transparency services [3]; pre-execution authorization records bound to request bytes [8]; human signatures over the exact action, including signed denials [9, 11]; refusal records as audit evidence [12]; and partial mappings to Articles 12, 14 and 19 [14, 17, 19].

To the author's knowledge, no existing specification composes these into a single profile in which:

- a human ratification is expressed as a COSE-signed SCITT Signed Statement bound to the digest of the exact canonical execution payload;
- the SCITT Registration Policy evaluated by the Transparency Service serves as the authorization gate, so that a Receipt is issued only for authorized actions;
- fail-closed, Receipt-gated execution is a normative requirement of the profile rather than a property of a particular deployment mode;
- the resulting evidence is related explicitly to Articles 12, 14, 19 and 26 of the EU AI Act together.

The Sovereign Acceptance Runtime addresses this composition gap. It is intended to complement the drafts above rather than compete with them; Section 7 discusses alignment.

## 3. Threat Model and Architectural Requirements

### 3.1 Security Objectives

The Sovereign Acceptance Runtime (SAR) is intended to ensure that autonomous actions executed by AI agents remain subject to cryptographically verifiable human authorization. Rather than treating approval as transient workflow state, the architecture seeks to establish durable authorization evidence that remains independently verifiable after execution.

The design is guided by five primary security objectives.

- **O1. Authorization Authenticity:** Every authorized action shall be attributable to an authenticated human overseer possessing an authorized signing credential.
- **O2. Authorization Integrity:** The authorization decision shall remain cryptographically bound to the exact payload reviewed by the human overseer.
- **O3. Evidence Portability:** Authorization evidence shall remain independently verifiable without requiring continued access to the original runtime environment.
- **O4. Execution Integrity:** Protected actions shall execute only after successful verification of all required authorization evidence.
- **O5. Audit Verifiability:** Independent auditors shall be capable of reconstructing and verifying the authorization chain using only published cryptographic artifacts.

These objectives define the security properties that the proposed architecture seeks to achieve and motivate the threat model described below.

### 3.2 Trust Model

The architecture involves four principal actors.

- **Agent (A):** An autonomous software component responsible for proposing actions. The agent is not inherently trusted.
- **Sovereign Overseer (S):** An authorized human responsible for reviewing proposed actions and issuing cryptographic ratifications or rejections.
- **Transparency Service (T):** A SCITT-compatible transparency service responsible for evaluating Registration Policies, recording accepted statements, and issuing cryptographic receipts.
- **Verifier (V):** An independent auditor, regulator, or third party validating authorization evidence after execution.

The architecture assumes:

- secure cryptographic primitives;
- uncompromised signing keys during authorization;
- correct implementation of signature verification algorithms.

No assumptions are made regarding:

- agent behaviour;
- runtime integrity;
- operating system trustworthiness;
- administrator honesty;
- network security.

Accordingly, the runtime itself is treated as potentially hostile until authorization evidence is successfully verified.

### 3.3 Adversary Model

Let $\mathcal{A}$ denote a probabilistic polynomial-time adversary capable of compromising application state, execution environments, communication channels, and operational infrastructure.

The adversary may:

- modify application memory;
- manipulate persistent storage;
- replay previously observed protocol messages;
- intercept network communication;
- fabricate application logs;
- attempt unauthorized action execution.

The adversary is not assumed capable of:

- breaking standard cryptographic primitives;
- forging valid signatures without possession of corresponding private keys;
- violating collision resistance of the selected hash algorithm.

### 3.4 Threats

The architecture considers six principal threats.

- **T1. Audit Log Modification:** The adversary modifies, deletes, or backdates operational records to conceal unauthorized actions or fabricate compliance evidence.
- **T2. Authorization Forgery:** The adversary injects fabricated approval events into application state or runtime logs without obtaining a valid sovereign signature.
- **T3. Authorization Repudiation:** A participant later disputes whether authorization occurred or claims that an observed authorization corresponds to a different action.
- **T4. Payload Substitution (TOCTOU):** The payload reviewed during authorization differs from the payload ultimately executed. This represents a classical time-of-check to time-of-use (TOCTOU) attack.
- **T5. Authorization Replay:** Previously valid authorization evidence is reused to authorize additional executions beyond its intended scope.
- **T6. Transparency Inconsistency:** A compromised transparency service presents inconsistent ledger views to different observers in an attempt to conceal unauthorized registrations.

### 3.5 Architectural Requirements

The identified threats motivate six architectural requirements.

- **R1. Cryptographic Ratification [Specified]:** Human authorization shall be represented by a cryptographic signature over a deterministic representation of the proposed action.
  - *Mitigates:* T2, T3
- **R2. Tamper-Evident History [Specified]:** Operational evidence shall be recorded using append-only cryptographic structures capable of revealing unauthorized modification.
  - *Mitigates:* T1
- **R3. Portable Authorization Evidence [Specified]:** Authorization artifacts shall remain independently verifiable without requiring access to the original runtime.
  - *Mitigates:* T3, T6
- **R4. Policy-Governed Registration [Specified]:** Registration shall succeed only after all authorization policies have been satisfied.
  - *Mitigates:* T2
- **R5. Fail-Closed Execution Binding [Specified]:** Execution shall proceed only after successful verification that the runtime payload exactly matches the ratified payload.
  - *Mitigates:* T4, T5
- **R6. Transparency Consistency [Specified]:** Registration evidence shall support independent detection of inconsistent transparency views.
  - *Mitigates:* T6

### 3.6 Threat Coverage

| Threat | Primary Requirement |
|---|---|
| T1 | R2 |
| T2 | R1, R4 |
| T3 | R1, R3 |

| Threat | Primary Requirement |
|---|---|
| T4 | R5 |
| T5 | R5 |
| T6 | R3, R6 |

The mapping establishes traceability between the adversarial model and the architectural mechanisms presented in the following section.

### 3.7 Scope of the Threat Model

The following scenarios are explicitly outside the scope of this work:

- Compromise of the sovereign overseer's private signing key prior to authorization.
- Failure of standard cryptographic primitives.
- Physical compromise of hardware security modules.
- Denial-of-service attacks preventing authorization or receipt issuance.
- Assessment of whether the human authorization decision itself was substantively correct.

These assumptions are common to many cryptographic authorization systems and represent areas for future investigation rather than objectives of the proposed architecture.

## 4. Sovereign Acceptance Runtime Architecture

### 4.1 Architectural Overview

The Sovereign Acceptance Runtime (SAR) is a deterministic authorization layer positioned between autonomous decision-making and external action execution.

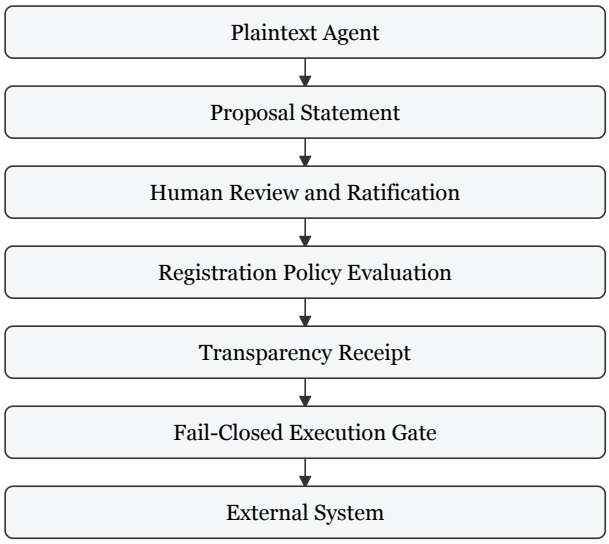
An agent may generate a proposed action, but it may not directly invoke a protected external operation. Before execution, the proposed action must pass through a sequence of cryptographic and policy checks that produce independently verifiable authorization evidence.

SAR separates the execution process into four distinct responsibilities:

1. **Action proposal**, performed by the autonomous agent.
2. **Human ratification**, performed by one or more authorized overseers.
3. **Policy evaluation and evidence registration**, performed by a transparency service.
4. **Execution authorization**, performed by the runtime immediately before invocation.

This separation prevents authorization from depending solely on mutable application state. Execution instead depends on cryptographic artifacts whose validity can be verified independently of the agent, application database, or user-interface workflow.

The core execution path is:




The architecture does not assume that the agent, application runtime, operating system, or network environment is trustworthy. A protected action is authorized only when the execution gate successfully validates the required signatures, policy context, transparency evidence, freshness constraints, and runtime payload binding.

## 4.2 Design Principles

SAR follows five design principles.

### *4.2.1 Explicit Authority*

No protected action is considered authorized unless an identified overseer has issued a valid cryptographic ratification. Silence, absence of rejection, workflow progression, or possession of application state does not constitute authorization.

### *4.2.2 Deterministic Verification*

Authorization decisions must be verifiable through deterministic procedures. Given the same proposal, ratification, policy, receipt, and trust anchors, independent verifiers should reach the same validation result.

### *4.2.3 Payload-Bound Ratification*

Human ratification applies to one deterministic representation of one proposed action. Authorization cannot be transferred to a modified payload without invalidating the cryptographic binding.

### *4.2.4 Portable Evidence*

Authorization evidence should remain verifiable outside the system in which the decision occurred. A regulator, auditor, counterparty, or incident investigator should not need access to the original runtime database to verify the authorization chain.

### *4.2.5 Fail-Closed Execution*

Execution is denied unless every required validation step succeeds. Missing, expired, malformed, unverifiable, or mismatched evidence is treated as authorization failure.

### 4.3 Running Example

To illustrate the architecture, consider an autonomous financial agent preparing a wire transfer.

The intended operation contains:

- a source account;
- a destination account;
- a transfer amount;
- a currency;
- a requested execution time;
- a transaction identifier.

In a conventional approval workflow, the agent may display this information to a human and resume execution after the user selects an approval button. The resulting authorization may survive only as application state or a conventional log record.

Under SAR, the agent first produces a canonical execution payload and a cryptographic Proposal Statement. The overseer reviews the exact canonical payload and signs a Ratification Statement referencing its digest. The proposal and ratification are evaluated against a Registration Policy. If the policy is satisfied, the transparency service registers the evidence and issues a Receipt.

Immediately before the banking API is invoked, the execution gate reconstructs the runtime payload, computes its digest, verifies the Receipt and ratification, and confirms that the runtime digest matches the ratified digest. The transfer proceeds only if every verification succeeds.

Although the example uses a financial transaction, the same model applies to infrastructure changes, access-control operations, industrial commands, healthcare workflows, data disclosure, and other actions requiring accountable human authorization.

### 4.4 Canonical Payload Representation

Cryptographic binding requires a deterministic representation of the proposed action. Let $P$ denote the canonical execution payload. $P$ includes all parameters whose modification could alter the external effect of the action.

For the running example:

```
P = {
    "action_type": "wire_transfer",
    "source_account": "...",
    "destination_account": "...",
    "amount": 5000,
    "currency": "EUR",
    "execution_time": "...",
    "transaction_id": "..."
}
```

The payload is encoded using deterministic Concise Binary Object Representation (CBOR) rules. Its digest is computed as:

$$d_P = H(P)$$

where $H$ is a collision-resistant cryptographic hash function, such as SHA-256.

Metadata that can alter execution meaning must be included in $P$. Descriptive fields that do not affect execution may be represented separately, but they must not be allowed to modify the action after ratification.

The security of the payload-binding mechanism depends on the canonical representation being stable. Two semantically identical actions must produce the same encoded representation, while any execution-relevant modification must produce a different digest.

**Status: [Proven]**

## 4.5 Protocol Objects

SAR defines four primary protocol objects:

- Proposal Statement;
- Ratification Statement;
- Registration Policy;
- Receipt and Transparent Authorization Record.

The notation in this section is illustrative. A future protocol profile would define normative Concise Data Definition Language (CDDL) schemas and validation procedures.

### *4.5.1 Proposal Statement*

A Proposal Statement represents an action that an agent seeks permission to execute. A minimal Proposal Statement contains:

```
ProposalStatement = {
    version: uint,
    proposal_id: text,
    agent_id: text,
    payload_digest: bytes,
    payload_media_type: text,
    action_type: text,
    target: text,
    issued_at: uint,
    expires_at: uint,
    nonce: bytes,
    policy_id: text
}
```

The Proposal Statement is signed by the agent using a COSE Single Signer Data Object (COSE_Sign1). Illustratively:

$$Proposal = \text{COSE_Sign1}_{K_A}(d_P, \text{metadata})$$

where $K_A$ is the agent signing key, $d_P$ is the digest of the canonical execution payload, and metadata contains the remaining proposal attributes.

The agent signature does not authorize execution. It identifies the source of the proposal and protects the proposal object from undetected modification.

**Status: [Partially Enforced]** (implemented with a subset of the illustrative fields; Section 6.2)

### *4.5.2 Ratification Statement*

A Ratification Statement records a sovereign overseer's decision regarding a specific Proposal Statement. A minimal Ratification Statement contains:

```
RatificationStatement = {
    version: uint,
    ratification_id: text,
    proposal_digest: bytes,
    proposal_id: text,
    overseer_id: text,
    verdict: text, ; "APPROVED" / "REJECTED"
    reviewed_at: uint,
    valid_until: uint,
    policy_id: text,
    reviewer_context: map
}
```

The verdict field contains an explicit authorization decision (`APPROVED` or `REJECTED`).

The overseer signs the Ratification Statement using COSE_Sign1:

$$Ratification = \text{COSE_Sign1}_{K_S}(H(Proposal), \text{verdict}, \text{policy_id})$$

where $K_S$ is the overseer's signing key.

The signature binds the overseer to the exact Proposal Statement rather than to an application-specific approval identifier. A rejection is represented as a signed artifact rather than as the absence of approval. This preserves evidence that human intervention occurred and records the decision that prevented execution.

**Status: [Partially Enforced]** (implemented with a subset of the illustrative fields; Section 6.2)

#### *4.5.3 Registration Policy*

A Registration Policy defines the conditions under which the transparency service may register an authorization record. A policy may specify:

```
RegistrationPolicy = {
    policy_id: text,
    policy_version: text,
    authorized_agents: [+ text],
    authorized_overseers: [+ text],
    permitted_action_types: [+ text],
    quorum: uint,
    proposal_validity_window: uint,
    ratification_validity_window: uint,
    replay_constraints: map,
    target_constraints: map,
    contextual_constraints: map
}
```

Typical policy requirements include:

- the proposal was signed by an authorized agent;
- the ratification was signed by an authorized overseer;
- the action type is permitted under the policy;
- the ratification references the submitted proposal;
- the required reviewer quorum is satisfied;
- the proposal and ratification are not expired;
- the transaction identifier or nonce has not previously been consumed;

- deployment-specific contextual conditions are satisfied.

The policy is identified by a versioned `policy_id`. Versioning is necessary because an auditor must be able to determine which policy governed an authorization at the time it was registered. The transparency service evaluates the policy before registration. A failed policy evaluation produces no authorization Receipt.

**Status: [Partially Enforced]** (evaluated by the Execution Gate rather than a Transparency Service; action-type, target and contextual constraints not implemented; Section 6.2)

#### *4.5.4 Receipt and Transparent Authorization Record*

When the Registration Policy is satisfied, the transparency service commits the accepted evidence to its append-only structure and issues a cryptographic Receipt.

The Receipt provides evidence that the submitted authorization record was accepted and incorporated into the transparency service. A Transparent Authorization Record consists of:

```
TransparentAuthorizationRecord = {
    proposal: ProposalStatement,
    ratification_set: [+ RatificationStatement],
    registration_policy_reference: text,
    receipt: bytes
}
```

Illustratively:

$$TAR = \langle Proposal, Ratifications, PolicyRef, Receipt \rangle$$

The Receipt is not itself a human approval. It proves that the proposal and ratification evidence were evaluated and registered under the identified policy. A verifier can use the Transparent Authorization Record to validate the authorization chain independently.

**Status: [Specified]**

### 4.6 Authorization Protocol

The SAR authorization protocol consists of five phases.

- **Phase 1: Proposal Generation** The agent constructs the canonical execution payload $P$, computes $d_P = H(P)$, and produces a signed Proposal Statement. The Proposal Statement is assigned a unique `proposal_id`, nonce, validity interval, and policy reference. At this stage, the action remains non-executable.
- **Phase 2: Human Review and Ratification** The overseer receives both:
  - a human-readable rendering of the intended action;
  - the canonical payload or a representation cryptographically linked to it. The interface must ensure that the reviewed representation corresponds to the payload digest in the Proposal Statement. After review, the overseer issues either a signed Ratification Statement with verdict `APPROVED` or a signed Ratification Statement with verdict `REJECTED`. A policy requiring multiple overseers produces a set of Ratification Statements.
- **Phase 3: Policy Evaluation** The proposal and ratification set are submitted to the transparency service. The service verifies:
  - proposal signature validity;
  - ratification signature validity;

- proposal-to-ratification binding;
- authorized agent and overseer identities;
- policy version;
- required quorum;
- permitted action type;
- validity intervals;
- nonce or transaction uniqueness;
- contextual policy constraints. Any validation failure terminates the registration attempt.

- **Phase 4: Registration and Receipt Issuance** If the Registration Policy is satisfied, the transparency service registers the authorization record within its append-only structure. The service then issues a Receipt containing the evidence necessary to verify registration and inclusion. Receipt issuance marks the transition from an approval decision to portable authorization evidence.
- **Phase 5: Gated Execution** Immediately before invoking the external system, the runtime execution gate receives:
  - the runtime payload $P_{\text{runtime}}$;
  - the Proposal Statement;
  - the required Ratification Statements;
  - the Registration Policy reference;
  - the Receipt. The gate verifies all authorization evidence and recomputes $d_{\text{runtime}} = H(P_{\text{runtime}})$. Execution is permitted only if:

$$\text{VerifyReceipt}(Receipt) = \text{TRUE}$$
$$\text{VerifyProposalSignature}(Proposal) = \text{TRUE}$$
$$\text{VerifyRatificationSignatures}(Ratifications) = \text{TRUE}$$
$$\text{EvaluatePolicy}(Proposal, Ratifications, Policy) = \text{TRUE}$$
$$d_{\text{runtime}} = d_P$$
$$\text{ReplayCheck}(proposal_id, nonce, transaction_id) = \text{PASS}$$

  If any condition fails, execution is denied.

**Status: [Partially Enforced]** (Phase 1 and Phase 5 implemented; Phase 2 implemented as signing only, without a review interface; Phase 3 performed by the Execution Gate; Phase 4, registration and Receipt issuance, not implemented)

### 4.7 Fail-Closed Execution Semantics

Fail-closed execution is the primary runtime enforcement property of SAR. The execution gate begins from the state DENY and transitions to ALLOW only after every mandatory validation succeeds.

The following conditions result in deterministic denial:

- Missing Proposal Statement;
- Invalid agent signature;
- Missing Ratification Statement;
- Invalid overseer signature;
- Insufficient approval quorum;
- Rejected verdict;
- Expired proposal;

- Expired ratification;
- Invalid or unrecognized policy version;
- Invalid Receipt;
- Payload digest mismatch;
- Previously consumed nonce or transaction identifier;
- Unavailable verification dependency where verification is mandatory.

The runtime does not fall back to application-level approval state if cryptographic verification fails. Timeouts, unavailable transparency evidence, parsing errors, and ambiguous policy results are treated as failures rather than as reasons to proceed. This behavior implements deny-by-default semantics at the boundary where the action acquires external effect.

**Status: [Partially Enforced]** (implemented and tested for all listed conditions except an invalid Receipt, an unrecognized policy version and an unavailable verification dependency)

### 4.8 Defense Against Authorization Forgery

Authorization forgery occurs when an attacker attempts to represent an action as approved without obtaining a valid sovereign signature.

SAR requires every approval decision to be encoded within a Ratification Statement signed by a policy-authorized overseer. An application database entry such as `approved = true` does not satisfy the Registration Policy and cannot produce a valid Receipt.

To forge authorization, an attacker would need to forge a valid overseer signature, compromise an authorized signing key, or cause the policy verifier to accept an unauthorized key. Under the cryptographic and trust assumptions in Section 3, a fabricated approval event cannot satisfy the ratification and registration checks. This mechanism directly addresses T2 through R1 and R4.

**Status: [Proven]**

### 4.9 Defense Against Payload Substitution

A time-of-check to time-of-use attack occurs when the payload presented during human review differs from the payload supplied to the external system.

SAR prevents this substitution by using the payload digest as the principal binding value throughout the authorization chain. The overseer ratifies a Proposal Statement containing $d_P$. Immediately before invocation, the execution gate recomputes $d_{\text{runtime}}$ from the actual execution payload.

Execution requires $d_{\text{runtime}} = d_P$. Any modification to an execution-relevant parameter changes the runtime digest and causes the gate to deny execution. For the running example, changing destination account, amount, currency, source account, execution identifier, or any other field included in the canonical payload after approval invalidates the authorization. This mechanism directly addresses T4 through R5.

**Status: [Proven]**

### 4.10 Replay Prevention

A valid authorization must not be reusable outside its intended execution context. SAR therefore requires each proposal to contain replay-relevant attributes, including at least:

- a unique `proposal_id`;
- a cryptographic nonce;

- an expiration time;
- an execution or transaction identifier.

The Registration Policy defines the applicable replay rule. A deployment may require that:

- each proposal be consumed at most once;
- each nonce be unique within a policy domain;
- each transaction identifier map to only one successful execution;
- authorization expire after a defined validity window;
- execution occur only at a specified target.

The execution gate records successful consumption of the authorization identifier before or atomically with invocation. A subsequent attempt to present the same authorization evidence fails the replay check. Where atomic execution and consumption cannot be guaranteed, the integration must use idempotency keys or an external transaction protocol appropriate to the target system. This mechanism directly addresses T5 through R5.

**Status: [Proven]** (for gates sharing one authorization store; Section 6.5)

### 4.11 Rejection and Interruption

A rejection is a signed governance event. When the overseer issues a Ratification Statement with verdict `REJECTED`, the transparency service may register the rejection for audit purposes, but it must not issue an authorization Receipt permitting execution.

A registered rejection provides evidence that a proposal was submitted, a human reviewed the proposal, authorization was explicitly denied, and the runtime was expected to remain in the deny state.

For actions already in progress, SAR distinguishes between pre-execution rejection, runtime interruption, and post-execution remediation. The core protocol governs pre-execution authorization. Runtime interruption requires an integration-specific stop mechanism capable of halting or containing an already initiated process. Accordingly, a signed rejection can authorize or document interruption, but its operational effect depends on the capabilities of the target system.

**Status: [Partially Enforced]** (signed rejection and denial implemented; registration of rejections and runtime interruption not implemented)

### 4.12 Independent Verification

An external verifier receives the Transparent Authorization Record and the relevant trust material. The verifier performs the following procedure:

1. Parse the Proposal Statement.
2. Verify the agent signature.
3. Reconstruct or obtain the canonical payload and confirm its digest.
4. Parse each Ratification Statement.
5. Verify each overseer signature.
6. Confirm that every ratification references the Proposal Statement.
7. Resolve the applicable Registration Policy and policy version.
8. Verify that the submitted evidence satisfied the policy.
9. Verify the Receipt and inclusion evidence.
10. Confirm validity intervals and replay-related identifiers.

11. Compare the registered payload digest with any independently obtained execution record.

The verifier does not need access to the original application database or user-interface state. The resulting verification establishes that a particular proposal and ratification set were registered under the identified policy. Additional execution evidence is required to establish that the external system actually performed the action.

SAR therefore distinguishes authorization evidence (proving execution was permitted) from execution evidence (proving execution occurred). The present architecture focuses primarily on authorization evidence, while allowing execution results to be linked as subsequent transparent statements.

**Status: [Partially Enforced]** (steps 1, 2 and 4–8 implemented, and the validity-interval part of step 10; payload reconstruction (step 3), Receipt verification (step 9) and comparison with execution records (step 11) not implemented)

### 4.13 Transparency Consistency

A transparency service may become malicious or compromised and attempt to present different ledger views to different verifiers. SAR relies on transparency mechanisms such as cryptographic inclusion proofs, consistency proofs, signed ledger roots, independent monitoring, and cross-verifier comparison.

A Receipt proves inclusion relative to a specific ledger state. Detection of persistent split views requires verifiers or monitors to compare signed roots or consistency evidence over time. Consequently, transparency receipts reduce dependence on the original runtime, but they do not eliminate the need for transparency monitoring. Requirement R6 is therefore fully specified at the architectural level but depends on deployment of appropriate consistency and monitoring mechanisms.

**Status: [Specified]**

### 4.14 Security Invariants

The security argument for SAR rests on five invariants.

- **I1. Proposal Uniqueness:** Every protected execution references one canonical Proposal Statement. The proposal identifies the intended payload through its cryptographic digest and replay-resistant identifiers.
- **I2. Ratification Binding:** Every valid Ratification Statement references one specific Proposal Statement or its digest. A ratification cannot be detached and applied to a different proposal without invalidating verification.
- **I3. Policy-Validated Registration:** A valid authorization Receipt is issued only after the proposal and ratification evidence satisfy the identified Registration Policy.
- **I4. Runtime Payload Equality:** Immediately before execution, the digest of the runtime payload must equal the digest contained in the authorized proposal.
- **I5. Deny-by-Default State:** The execution gate remains in the deny state unless all required verification procedures succeed.

### 4.15 Security Argument

Under the assumptions defined in Section 3, the invariants support the following security claim: A protected autonomous action cannot reach execution through the SAR gate unless valid authorization evidence exists for the exact runtime payload, the applicable Registration Policy is satisfied, and the authorization has not expired or been consumed.

The claim follows from these mechanisms:

- **Authorization forgery:** An attacker without an authorized overseer key cannot produce a valid Ratification Statement satisfying I2 and I3. Fabricated workflow state therefore cannot authorize execution.
- **Payload substitution:** A substituted payload violates I4 because the recomputed runtime digest differs from the digest in the ratified proposal.
- **Authorization replay:** Reuse of authorization evidence violates the policy-defined replay constraints enforced under I3 and the execution checks associated with I5.
- **Log modification:** Modification of registered authorization evidence invalidates signatures, receipts, hash relationships, or consistency proofs.
- **Repudiation:** The overseer's signed Ratification Statement and associated Receipt provide independently verifiable evidence of the recorded authorization decision, subject to key-management assumptions.
- **Transparency inconsistency:** Conflicting ledger states may be detected through signed roots, inclusion proofs, consistency proofs, and independent monitoring.

This argument is architectural rather than a formal cryptographic proof. A normative security proof would require a fully specified protocol, formal adversary game, canonical encoding rules, and implementation-independent verification semantics.

### 4.16 Requirement Traceability

| Requirement | Primary SAR Mechanism |
|---|---|
| **R1. Cryptographic Ratification** | COSE-signed Ratification Statement bound to the Proposal Statement |
| **R2. Tamper-Evident History** | Append-only transparency registration and cryptographic receipts |
| **R3. Portable Authorization Evidence** | Transparent Authorization Record and offline verification |
| **R4. Policy-Governed Registration** | Versioned machine-enforceable Registration Policy |
| **R5. Fail-Closed Execution Binding** | Pre-invocation receipt, signature, replay, and payload-digest verification |
| **R6. Transparency Consistency** | Inclusion proofs, signed roots, consistency evidence, and monitoring |

## 5. Architectural Correspondence to the EU AI Act

### 5.1 Scope of the Regulatory Analysis

The objective of this section is not to determine legal compliance with Regulation (EU) 2024/1689 (the EU AI Act), nor to provide legal advice. Rather, it examines the extent to which the architectural mechanisms introduced by the Sovereign Acceptance Runtime (SAR) correspond to selected technical objectives expressed in the Regulation.

The EU AI Act establishes legal obligations while remaining largely technology-neutral. It specifies what high-risk AI systems must achieve but generally does not prescribe how these objectives should be implemented. Consequently, multiple technical architectures may satisfy the same regulatory objective.

SAR is presented as one possible architectural approach for implementing verifiable authorization and evidentiary traceability within autonomous AI systems.

### 5.2 Architectural Correspondence

| EU AI Act Provision | Regulatory Objective | SAR Architectural Mechanism | Implementation State |
|---|---|---|---|
| Article 12 | Automatic recording of operational events | Proposal Statements, Ratification Statements, Receipts, append-only transparency registration | [Partially Enforced] |
| Article 14 | Technical human oversight | Cryptographic human ratification evaluated before execution | [Partially Enforced] |
| Article 14(4) | Human ability to intervene or stop operation | Signed rejection, policy denial, fail-closed execution gate | [Partially Enforced] |
| Article 19 | Provider retention of operational logs | Portable authorization evidence suitable for long-term storage | [Partially Enforced] |
| Article 26 | Deployer operational accountability and log retention | Verifiable authorization records supporting deployment governance | [Specified] |

The correspondence identifies architectural relationships rather than demonstrating regulatory compliance. Implementation states refer to the reference implementation of Section 6; Receipts and transparency registration are not yet implemented. For high-risk systems, the obligations in Articles 12, 14, 19 and 26 apply from 2 December 2027 (Annex III) and 2 August 2028 (Annex I) [2, Art. 1(40)].

### 5.3 Article 12 – Automatic Event Recording

Article 12 requires high-risk AI systems to record operational events throughout their lifecycle in a manner supporting traceability.

SAR addresses this objective by representing each authorization decision as a sequence of cryptographically related protocol objects: Proposal Statement, Ratification Statement, Registration Policy reference, and Transparency Receipt. Unlike conventional application logs, these artifacts are designed to remain independently verifiable after execution.

Importantly, the Regulation requires automatic recording but does not prescribe append-only transparency structures, cryptographic receipts, or evidentiary verification mechanisms. SAR introduces these architectural mechanisms as one possible implementation strategy rather than as regulatory requirements.

### 5.4 Article 14 – Human Oversight

Article 14 requires that human oversight be technically supported throughout operation of high-risk AI systems. Conventional human-in-the-loop mechanisms typically satisfy this objective by requiring user interaction before execution.

SAR adopts a stronger architectural interpretation in which human authorization becomes a cryptographically verifiable protocol artifact rather than transient workflow state. Within SAR, approval is represented by a signed Ratification Statement, authorization is evaluated before execution, execution

depends on successful policy verification, and unsuccessful verification prevents execution through a fail-closed execution gate. Accordingly, oversight becomes an enforceable runtime property rather than solely an interaction pattern.

### 5.5 Article 14(4): Human Intervention

Article 14 further requires that natural persons remain capable of intervening in system operation where appropriate.

SAR distinguishes three forms of intervention: authorization before execution, rejection preventing execution, and interruption mechanisms provided by deployment-specific integrations. The core protocol governs pre-execution authorization. Operational interruption of actions already underway depends on the capabilities of the external system and therefore remains outside the protocol itself. Consequently, SAR supports technical intervention before execution while allowing deployment-specific interruption mechanisms to be incorporated where required.

### 5.6 Articles 19 and 26 – Operational Records

The Regulation requires providers and deployers to retain operational records for defined periods.

SAR does not replace organizational record-retention policies. Instead, it defines portable cryptographic authorization artifacts that may be retained alongside existing operational records. Because Proposal Statements, Ratification Statements, and Receipts remain independently verifiable, they may strengthen the evidentiary quality of retained authorization records without altering the underlying retention obligations.

### 5.7 Relationship to Harmonized Standards

At the time of writing, harmonized technical standards addressing AI logging and human oversight continue to evolve. Current draft work identifies important architectural directions but does not yet define interoperable protocol semantics for cryptographically verifiable human authorization prior to autonomous execution. The deferral of the high-risk obligations to December 2027 and August 2028 [2] leaves a window in which such semantics could mature, both in European standardisation and in the IETF drafts discussed in Section 2.6.

SAR therefore should be viewed as complementary to ongoing standardization efforts rather than as an alternative to them. The proposed architecture intentionally reuses existing Internet standards—including SCITT and COSE—to maximize future interoperability should standardized authorization profiles emerge.

### 5.8 Regulatory Interpretation Boundaries

Several observations should be emphasized:

- The EU AI Act does not require authorization records to be immutable or stored in transparency ledgers.
- The Regulation does not prescribe SCITT, COSE, CBOR, Merkle proofs, or any other specific cryptographic technology.
- Implementing SAR alone does not establish compliance with Regulation (EU) 2024/1689.

Compliance ultimately depends upon the complete AI system, organizational governance, applicable harmonized standards, sector-specific regulation, and the assessment of competent supervisory authorities. Accordingly, the contribution of SAR is architectural rather than legal.

### 5.9 Summary

The analysis presented in this section demonstrates that the mechanisms introduced by the Sovereign Acceptance Runtime correspond to several technical objectives expressed in the EU AI Act while remaining independent of any particular legal interpretation.

Rather than implementing the Regulation itself, SAR illustrates how existing transparency and signing technologies may be composed into an execution architecture capable of producing portable, cryptographically verifiable authorization evidence. This distinction preserves both technical neutrality and regulatory flexibility while providing a concrete foundation for the implementation of accountable autonomous AI systems.

## 6. Reference Implementation and Evaluation

### 6.1 Evaluation Objectives

The reference implementation is not intended to demonstrate production readiness. Its purpose is to establish whether the core authorization architecture of Section 4 can be realized in software while preserving the security properties of Section 3, and to make every implementation claim in this paper testable by third parties. The evaluation addresses three research questions:

- **RQ1.** Can protected actions be prevented from executing unless valid authorization evidence is presented?
- **RQ2.** Can modification of the execution payload after human ratification be detected before execution?
- **RQ3.** Can authorization evidence be verified independently of the runtime that generated it?

### 6.2 Implementation

The reference implementation, *sar-cose* [23], is a single Python module of about 430 lines. It depends only on the `cbor2` and `cryptography` libraries.

- **Statements.** Proposal and Ratification Statements are tagged COSE_Sign1 objects [5]. The protected header carries the algorithm, the content type (distinct for proposals and ratifications) and the signer's key identifier; unprotected header parameters are rejected.
- **Encoding.** Payloads use the core deterministic CBOR encoding [4, Section 4.2.1]. The encoder is checked against the examples in Appendix A of [4], and the decoder rejects any input that is not in deterministic form.
- **Signatures.** Signatures use Ed25519 under the fully specified algorithm identifier $-19$ [7]; the deprecated polymorphic identifier $-8$ is rejected by default.
- **Binding.** The payload digest is SHA-256 over the deterministic encoding. A ratification binds to the SHA-256 digest of the exact COSE_Sign1 bytes of the proposal.
- **Execution Gate.** Before allowing an action, the gate:
  - evaluates the Registration Policy: authorized agents and overseers, a quorum of distinct approving overseers, policy binding of both statement types, proposal and ratification validity windows, and a bound on future-dated statements;
  - verifies both statement types;
  - compares the runtime payload digest with the ratified digest;
  - consumes the proposal's nonce atomically with recording the ALLOW decision.

  Every failure, including any malformed input, results in denial.

- **Authorization Store.** The store is an SQLite hash chain of decisions. Its head is signed as a checkpoint after every commit, and writers take the database write lock before reading the chain tip, so several gates may share one database.
- **Offline verification.** A stateless verifier checks that a set of artifacts constitutes a valid authorization under a given policy, using only the artifacts and the policy's trust anchors.

The following parts of Section 4 are **not implemented**:

- registration with a SCITT Transparency Service, and issuance and verification of Receipts and inclusion proofs (Sections 4.5.4, 4.6 Phase 4, 4.13): Registration Policy evaluation currently runs inside the gate;
- publication of store checkpoints outside the store;
- the full field set of the illustrative CDDL in Section 4.5 (for example `payload_media_type`, `target`, `reviewer_context`).

### 6.3 Conformance Suite

The implementation is evaluated against an implementation-independent conformance suite of 55 automated tests in four groups.

- **Original scenarios (12 tests).** These cover the scenarios of the original evaluation: authorized execution, missing authorization, invalid signature, rejected proposal, modified payload after approval, replayed authorization, expired proposal, expired ratification and invalid policy configuration. They also cover offline verification, tamper evidence and quorum. These tests were written before the implementation and are used unmodified.
- **Gap tests (11 tests).** These were added after a mutation-based review showed that the original tests did not detect certain defects. They cover:
  - future-dated ratifications, and ratifications dated before their proposal (C07c);
  - an expired ratification reviewed after its proposal (C07d);
  - offline verification of records with no ratification, only a rejection, or a quorum shortfall (C09b);
  - deletion of the most recent log record (C10b);
  - a quorum "met" by one overseer twice (C11b);
  - statements bound to another policy (C12);
  - an approval presented with a different proposal (C13).

  The harness's illustrative stub implementation fails 9 of these 11 tests, which shows that they detect the defects they target.
- **COSE- and CBOR-specific tests (16 tests).** These cover:
  - the Appendix A vectors and key ordering of [4];
  - the COSE_Sign1 structure;
  - statements signed under a deprecated algorithm, tampered protected headers, a key identifier that does not match the signer, and content-type (role) confusion;
  - non-deterministic encodings (unsorted or duplicate map keys, non-preferred integers, indefinite lengths) and unprotected header parameters;
  - cross-library verification. Statements are verified by independent code built on `cbor2` and `cryptography`, and in both directions with the `pycose` library. Because `pycose` does not implement algorithm $-19$, the `pycose` checks use statements signed under $-8$, which the gate itself rejects.
- **Property-based and system tests (16 tests).** These are summarized in Section 6.5.

### 6.4 Mutation Analysis

A passing test suite is evidence for a property only if the suite would fail without the mechanism that provides it. The implementation therefore exposes 19 *mutants*, each of which disables exactly one guard: payload-digest comparison, replay consumption, proposal expiry, ratification expiry, signature verification, rejection handling, chain verification, distinct-overseer counting, policy binding, time-window checks, offline quorum, checkpoint verification, algorithm, content-type and key-identifier checks, deterministic-encoding enforcement, ratification binding at the gate and offline, and write-transaction locking. **All 19 mutants are killed**: with any one guard removed, at least one test fails.

Building and testing the implementation found one gap in the test suite and one defect in the implementation.

- **Ratification binding was untested.** With the gate's ratification-to-proposal binding check removed, all 38 tests then in the suite still passed. The original scenario set never presents an approval issued for one proposal together with a different proposal, so invariant I2 was not covered. Test C13 was added.
- **Concurrent writers forked the log.** A concurrency test showed that, although replay protection held, concurrent writers sharing a database could append to the same chain tip and fork the hash chain. Writers now take the write lock before reading the tip. The corresponding mutant is killed in 10 of 10 runs, and the fixed implementation passed 20 of 20 runs.

### 6.5 Results

**RQ1 — Authorization enforcement.** The gate was compared with an independent model of the authorization rules of Sections 4.5–4.10, written as a plain rule list separately from the gate's code, in two ways:

- **Random scenarios.** 3,000 random scenarios varying quorum, timing, signatures, binding, verdicts, policy and payload. The gate agreed with the model in all of them (182 allowed, 2,818 denied).
- **Single-fault scenarios.** 2,000 scenarios, each a valid scenario with zero or one of 16 injected faults. Again there were no disagreements: all 557 fault-free scenarios were allowed, and each of the 14 fault types that must always cause denial did so in every instance.

In every scenario, a second presentation of the same authorization was denied. In 16 concurrent gates sharing one database and presenting the same authorization simultaneously, exactly one presentation was allowed in each of five rounds, and replay protection persisted across a store restart.

- **Result: Supported** for gates sharing one authorization database.

**RQ2 — Payload integrity.** Property-based tests generated random payloads and changed, removed or added any field after approval. Execution was denied unless the deterministic encoding was byte-identical; the insertion order of map keys had no effect. Strings that differ only in Unicode normalization are treated as different actions, because binding is byte-exact. All single-bit flips of a signed proposal and ratification (over 3,000 variants) were rejected, either when parsing or at the gate.

- **Result: Supported.**

**RQ3 — Independent verification.** The offline verifier accepts valid records and rejects modified ones, records without ratification, records containing a rejection, records below quorum, records bound to another policy, and approvals moved to another proposal. Statements produced by the implementation are verified by independent code and by the `pycose` library, and statements produced by `pycose` are verified by the implementation. Verification of Transparency Service Receipts is not implemented.

- **Result: Supported for signed statements; not evaluated for Receipts.**

**Indicative performance.** On an Apple M5 Pro (single thread, Python 3.14), the run recorded in the repository's `RESULTS.md` gave the following median costs:

- creating a signed statement: about 40 $\mu$s;
- offline verification of a record: about 0.14 ms;
- gate authorization: about 0.2 ms with an in-memory store, and about 1.3 ms with a file-backed store (99th percentile about 17 ms, dominated by synchronous disk writes).

Repeated runs on the same machine varied by a factor of about two with system load. These figures indicate that the cryptographic overhead is small relative to human review latency; they are not a performance evaluation.

### 6.6 Security Properties Observed

| Property | Implementation State | Evidence |
|---|---|---|
| Proposal authenticity | [Proven] | Signature, algorithm, key-identifier and content-type tests; bit flips; mutants killed |
| Ratification authenticity | [Proven] | As above; cross-library verification |
| Ratification binding (I2) | [Proven] | C13; differential tests; mutants killed (gate and offline) |
| Payload binding (I4) | [Proven] | Modified-payload and property-based payload tests; mutant killed |
| Fail-closed execution (I5) | [Proven] | Differential and single-fault tests; fuzzed inputs never allowed |
| Offline verification | [Proven] for signed statements | C09b, C12, C13; cross-library tests. Receipts: [Specified] |
| Replay detection | [Proven] for gates sharing one database | Replay invariant in 5,000 scenarios; concurrency and restart tests |
| Tamper-evident history | [Partially Enforced] | Modification and truncation detected, but the checkpoint is held by the store rather than published externally |
| Transparency registration, Receipts, consistency (R6) | [Specified] | Not implemented |

All states refer to the reference implementation described in Section 6.2 and are reproducible with the published test suite (see Data and Code Availability).

### 6.7 Limitations of the Evaluation

The evaluation does not establish production readiness. It does not cover:

- a SCITT Transparency Service, Receipts, or interoperability between independent SCITT implementations;
- replay protection across gates that do not share a database;
- hardware-backed keys, key rotation or revocation;
- threshold cryptography;

- usability of the overseer's review interface, including whether the rendered action matches the signed payload;
- production-scale performance;
- resilience against denial-of-service attacks.

Cross-library interoperability was tested only with the deprecated algorithm identifier, because the available library does not yet support $-19$. Passing the suite shows that the specified guards exist and are exercised; it does not show the absence of other defects, and it is not a substitute for an independent security review.

### 6.8 Relationship to the Security Argument

| Invariant | Tests | Prototype Evidence |
|---|---|---|
| I1 Proposal Uniqueness | Replay tests; replay invariant in differential tests; concurrency | Verified within one shared store |
| I2 Ratification Binding | 03, 09, C13; differential tests | Verified |
| I3 Policy-Validated Registration | 08, 11, C11b, C12; differential tests | Verified for gate-side policy evaluation; not at a Transparency Service |
| I4 Runtime Payload Equality | 05; payload properties; bit flips | Verified |
| I5 Deny-by-Default State | 02, 04, 07a–d; single-fault tests; fuzzing | Verified |

The security argument of Section 4.15 remains architectural. The tests provide implementation evidence for invariants I1–I5 within the stated scope; they are not a formal proof.

### 6.9 Summary

The reference implementation shows that COSE-signed human ratification, payload-bound and policy-governed authorization, and fail-closed execution can be realized with existing libraries at negligible computational cost. Its claims rest on a public conformance suite whose adequacy is itself checked by mutation analysis. The principal remaining work is integration with a SCITT Transparency Service, so that Receipts become the execution gate as specified in Section 4.

## 7. Standardization Path

### 7.1 Motivation

The objective of the Sovereign Acceptance Runtime is not to introduce a new cryptographic primitive, transparency service, or authorization protocol. Instead, SAR proposes how existing Internet standards may be composed into an interoperable architecture for verifiable human authorization in autonomous AI systems.

Accordingly, SAR should be understood as an architectural profile built upon existing standards rather than a competing protocol stack. This design philosophy minimizes ecosystem fragmentation while allowing existing implementations of transparency services, signed statements, and verification libraries to remain applicable.

### 7.2 Architectural Position

The proposed architecture intentionally separates responsibilities between existing standards and the additional semantics introduced by SAR.

- SCITT remains responsible for transparency services, statement registration, cryptographic receipts, and independent verification.
- COSE remains responsible for digital signatures, cryptographic authenticity, and protected statement encoding.
- CBOR remains responsible for deterministic binary serialization.

SAR introduces only the missing authorization semantics required for autonomous execution: Proposal Statements, Ratification Statements, Registration Policies, execution binding, and fail-closed authorization semantics.

Accordingly, SAR extends existing standards at the semantic layer without modifying their underlying cryptographic mechanisms.

SAR should also align with the concurrent drafts discussed in Section 2.6 rather than duplicate them. In particular, the Permit profile [8] already defines pre-execution records bound to request bytes, and the authorization-receipt drafts [9, 10] already define human signatures over the exact action and signed denials. A SAR profile would ideally reuse their binding and receipt semantics where compatible. Its own contribution would then be limited to expressing human ratification as a SCITT Signed Statement and to the Registration Policy semantics that make a Receipt the gate for execution.

### 7.3 Proposed Standardization Strategy

The proposed standardization path consists of four incremental stages.

- **Stage 1 — Informational Architecture** Publication of an informational document describing the threat model, protocol objects, architectural invariants, security objectives, and authorization workflow. This paper represents that first stage.
- **Stage 2 — Experimental Profile** An experimental specification would define interoperable protocol semantics, including CDDL schemas, mandatory object fields, canonical serialization rules, signature profiles, receipt validation procedures, and Registration Policy semantics. It would most naturally take the form of an individual Internet-Draft submitted for discussion in the SCITT Working Group, building on RFC 9943 [3] and the drafts in Section 2.6. Independent implementations could then validate interoperability.
- **Stage 3 — Interoperability** Multiple independent implementations should demonstrate object compatibility, receipt interoperability, policy portability, and verifier consistency. At this stage, interoperability events and implementation reports become more valuable than additional architectural refinement.
- **Stage 4 — Community Standardization** If sufficient implementation experience is obtained, elements of the architecture could be proposed for broader standardization through existing Internet standards processes. Whether this occurs depends upon community adoption, implementation experience, and consensus rather than the existence of the present work.

### 7.4 Candidate Specification Components

A future interoperable profile would likely define several normative components:

- **Protocol Objects:** Normative definitions of Proposal Statement, Ratification Statement, Registration Policy, and Transparent Authorization Record.

- **CDDL Schemas:** Machine-readable schemas defining mandatory fields, optional extensions, version negotiation, and forward compatibility.
- **Registration Policy Language:** A portable policy vocabulary describing authorized identities, quorum rules, replay constraints, validity intervals, and execution contexts.
- **Verification Procedures:** Normative algorithms for Proposal verification, Ratification verification, Receipt verification, policy evaluation, and execution authorization.

### 7.5 Interoperability Goals

The long-term objective is interoperability rather than implementation uniformity. Independent implementations should be capable of exchanging authorization artifacts while remaining free to differ in programming language, storage engine, transparency infrastructure, deployment model, and policy implementation. Only the externally observable protocol semantics require interoperability.

### 7.6 Relationship to Model Context Protocol

One potential deployment model involves middleware for the Model Context Protocol (MCP). Rather than modifying foundation models or application logic, an SAR middleware component may intercept tool invocations requiring protected execution.

The middleware performs proposal generation, human ratification, Registration Policy evaluation, transparency registration, Receipt verification, and fail-closed execution. Under this approach, existing AI agents require minimal modification because authorization is introduced as an execution-layer concern rather than an application-layer feature.

### 7.7 Research Opportunities

The proposed architecture suggests several avenues for future research, including threshold ratification using distributed approval, delegated authorization, hardware-backed sovereign credentials, confidential transparency services, zero-knowledge authorization proofs, cross-organizational transparency federation, policy synthesis using formal methods, and automated verification of Registration Policies.

### 7.8 Summary

SAR is intended to evolve through incremental implementation experience rather than immediate standardization. By building upon existing Internet standards—including SCITT, COSE, and CBOR—the architecture minimizes disruption while introducing a narrowly scoped semantic extension for cryptographically verifiable human authorization.

## 8. Discussion and Limitations

### 8.1 Principal Contribution

The central contribution of the Sovereign Acceptance Runtime is not a new cryptographic primitive, transparency service, or digital signature algorithm. Instead, it proposes an execution architecture in which human authorization becomes a first-class security artifact rather than transient application state.

The architecture combines existing standards for cryptographic signing and transparency into a deterministic authorization pipeline that establishes an explicit relationship between four elements: the intended action, the human authorization decision, the governing authorization policy, and the resulting execution. By treating authorization as independently verifiable evidence rather than as workflow metadata, SAR shifts human oversight from an application-level interaction toward an architectural security property.

### 8.2 Relationship to Existing Approaches

The proposed architecture complements rather than replaces existing governance technologies. Traditional approval workflows successfully coordinate operational processes by allowing users to approve or reject actions before execution, but authorization generally remains embedded within application state or workflow history. Software supply-chain systems such as Sigstore and in-toto demonstrate how cryptographically verifiable evidence can improve software provenance and artifact integrity. SCITT extends these ideas by providing interoperable transparency services for signed statements.

SAR differs in objective: rather than establishing provenance of software artifacts, it establishes provenance of authorization decisions governing autonomous execution. It shares that objective with the concurrent drafts in Section 2.6 [8–17]. Its distinguishing element is the composition described in Section 2.7, not any individual mechanism.

### 8.3 Architectural Trade-offs

The architecture intentionally introduces additional operational complexity in exchange for stronger authorization guarantees.

- **Authorization Latency:** Human ratification necessarily introduces delay before execution. While acceptable for high-risk actions, identical mechanisms would be inappropriate for every autonomous operation. Registration Policies should therefore distinguish between actions requiring explicit human authorization and those suitable for automated execution.
- **Operational Complexity:** Compared with conventional approval workflows, SAR introduces cryptographic key management, policy administration, receipt verification, transparency infrastructure, and independent verification procedures. These components increase operational complexity but also improve evidentiary quality.
- **Policy Design:** The correctness of authorization ultimately depends upon Registration Policies. Poorly designed policies may authorize excessive actions, require unnecessary approvals, create reviewer fatigue, or reduce operational efficiency. SAR therefore provides enforcement mechanisms rather than guaranteeing policy quality.

### 8.4 Limitations

- **Key Management:** The architecture assumes secure management of sovereign signing credentials. Compromise of an authorized signing key allows unauthorized ratification, whereas loss of required signing keys may prevent authorization entirely. Future work should investigate hardware-backed credentials, delegated authorization, threshold signatures, and organizational key recovery.
- **Human Decision Quality:** SAR verifies that authorization occurred; it does not evaluate whether the human decision itself was appropriate. A correctly signed but incorrect authorization remains cryptographically valid. Consequently, the architecture addresses evidentiary integrity rather than human judgment.
- **Implementation Maturity:** The reference implementation covers the authorization path but not Transparency Service integration, Receipts or consistency monitoring (Section 6.2). Replay protection is evaluated only for gates sharing one store.
- **Transparency Infrastructure:** The planned evaluation targets the authorization pipeline rather than production transparency services. Large-scale deployment introduces additional questions concerning service availability, consistency monitoring, federation, and operational governance.

- **Performance:** Section 6.5 reports indicative single-thread costs only; throughput, contention and latency under realistic deployment were not measured. Future evaluation should measure authorization latency, throughput, receipt generation cost, verification overhead, and scalability.
- **Regulatory Interpretation:** The architectural correspondence presented in Section 5 should not be interpreted as legal guidance. Future harmonized standards, judicial interpretation, and supervisory practice may influence how technical mechanisms are evaluated within regulatory contexts.

### 8.5 Threats to Validity

- **Internal Validity:** Results from a single reference implementation will reflect that implementation; cross-implementation verification (Section 6.4) is required to separate protocol properties from implementation choices.
- **External Validity:** The planned evaluation runs in a controlled environment. Industrial deployment may introduce additional integration challenges.
- **Construct Validity:** Passing the conformance suite would demonstrate implementation of the proposed authorization model; it would not establish superiority over existing governance architectures or over the concurrent drafts in Section 2.6.
- **Ecological Validity:** Human oversight behaviour, reviewer fatigue, organizational governance, and operational policy evolution were not evaluated.

### 8.6 Future Research

Several research directions naturally emerge from the proposed architecture, including threshold human authorization, delegated sovereign identities, confidential transparency services, zero-knowledge authorization receipts, decentralized policy distribution, automated policy verification, interoperability testing across independent SCITT implementations, formal verification of authorization semantics, and empirical studies of operational human oversight.

### 8.7 Discussion

The increasing autonomy of AI systems changes the role of authorization within software systems. Historically, authorization primarily determined who was permitted to invoke an operation. For autonomous AI agents, authorization increasingly determines whether independently acting software should be permitted to perform irreversible actions whose consequences extend beyond the computing environment itself.

This shift motivates treating authorization not merely as application state but as portable, cryptographically verifiable evidence. The Sovereign Acceptance Runtime represents one architectural response to that transition. By combining existing transparency and signing technologies into a deterministic authorization pipeline, it proposes that human oversight can be expressed as verifiable protocol semantics rather than solely as user-interface interaction. The growing number of concurrent drafts on the same problem suggests the question is timely.

## 9. Conclusion

Autonomous AI systems are increasingly capable of producing irreversible effects outside the computing environments in which they operate. As this transition continues, the architectural role of human oversight changes accordingly. Approval mechanisms that were originally designed as workflow interactions become security-critical components whose evidentiary quality directly affects accountability, auditability, and trust.

This paper has argued that existing human-in-the-loop mechanisms generally provide operational control but rarely produce portable cryptographic evidence demonstrating who authorized an action, which exact execution payload was reviewed, and whether the executed payload remained identical to the approved payload. At the same time, the EU AI Act establishes regulatory objectives concerning operational traceability and human oversight while intentionally remaining technology-neutral regarding implementation.

To address this architectural gap, the paper introduced the Sovereign Acceptance Runtime (SAR), an execution architecture that models human authorization as a cryptographically verifiable ratification evaluated before protected actions are executed. Rather than introducing new cryptographic primitives, SAR composes existing Internet standards—including SCITT, COSE, and CBOR—into an authorization pipeline that combines signed proposals, human ratifications, policy-governed registration, transparency receipts, and fail-closed execution.

The work made four principal contributions:

1. It developed an adversarial threat model for autonomous execution and derived six architectural requirements governing verifiable human ratification.
2. It proposed an authorization architecture that binds human decisions to deterministic execution payloads through machine-enforceable Registration Policies evaluated before execution.
3. It provided a reference implementation of COSE-signed ratification, policy evaluation, payload-bound fail-closed execution and offline verification, evaluated against an implementation-independent conformance suite whose adequacy was checked by mutation analysis. The process exposed a gap in the original test scenarios (ratification binding) and a concurrency defect in the implementation, both of which were corrected.
4. It outlined a potential path toward interoperability by positioning SAR as an architectural profile built upon existing Internet standards rather than as a replacement for them.

The proposed architecture is intentionally limited in scope. It does not claim to establish legal compliance with Regulation (EU) 2024/1689, nor does it prescribe a universal implementation of human oversight. Instead, it illustrates one possible technical realization of independently verifiable authorization using existing transparency and signing technologies. Future harmonized standards, implementation experience, and operational deployments will ultimately determine which architectural approaches become widely adopted.

As AI systems continue to evolve from advisory tools into autonomous actors, the mechanisms governing authorization become increasingly significant. Treating authorization as portable cryptographic evidence rather than transient workflow state represents one possible direction for improving accountability without requiring fundamental changes to existing Internet security standards.

---

**Acknowledgements**
The author thanks the members of the IETF Supply Chain Integrity, Transparency, and Trust (SCITT) Working Group and the broader Internet security community for publicly developing the standards that inspired this work. Any errors, interpretations, or architectural proposals presented in this paper remain solely the responsibility of the author.

**Conflict of Interest**
The author declares no competing financial interests related to this work.

**Data and Code Availability**

The reference implementation (sar-cose), the conformance suite, the mutation-analysis scripts, the benchmark and the raw results reported in Section 6 are published under the Apache License 2.0 at https://github.com/rnfck88qng-a11y/sar-cose and archived on Zenodo as release v1.1.0, doi:10.5281/zenodo.23270796 [23]. The results reported in Section 6 correspond to that release. Properties tagged [Proven] are reproducible by running `run_tests.sh` and `run_mutants.sh` on it.

**Author Contributions**

The work was conceived, designed, implemented, and written by the sole author.

**Funding**

This research received no external funding.

**Disclaimer**

This paper presents an architectural proposal for cryptographically verifiable authorization in autonomous AI systems. Nothing in this paper should be interpreted as legal advice or as a definitive interpretation of Regulation (EU) 2024/1689. Regulatory compliance depends upon the complete AI system, deployment context, applicable legislation, future harmonized standards, and the assessment of competent supervisory authorities. The architectural correspondence discussed throughout this work is intended solely as technical analysis.

---

## References

URLs were retrieved on 9 October 2026. Internet-Drafts are works in progress; the version current at that date is cited.

[1] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). OJ L, 2024/1689, 12.7.2024. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[2] Regulation (EU) 2026/1744 of the European Parliament and of the Council of 8 July 2026 amending Regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as regards the simplification of the implementation of harmonised rules on artificial intelligence (Digital Omnibus on AI). OJ L, 2026/1744, 24.7.2026. https://eur-lex.europa.eu/eli/reg/2026/1744/oj

[3] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, S. Lasker. An Architecture for Trustworthy and Transparent Digital Supply Chains. RFC 9943, June 2026. https://www.rfc-editor.org/rfc/rfc9943

[4] C. Bormann, P. Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, December 2020. https://www.rfc-editor.org/rfc/rfc8949

[5] J. Schaad. CBOR Object Signing and Encryption (COSE): Structures and Process. RFC 9052, August 2022. https://www.rfc-editor.org/rfc/rfc9052

[6] J. Schaad. CBOR Object Signing and Encryption (COSE): Initial Algorithms. RFC 9053, August 2022. https://www.rfc-editor.org/rfc/rfc9053

[7] M.B. Jones, O. Steele. Fully-Specified Algorithms for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE). RFC 9864, October 2025. https://www.rfc-editor.org/rfc/rfc9864

[8] C. Munoz. A SCITT Profile for Pre-Execution AI Action Authorization Records. Internet-Draft draft-munoz-scitt-permit-profile-01, July 2026 (-00: May 2026). https://datatracker.ietf.org/doc/draft-munoz-scitt-permit-profile/

[9] I. Schrock. Authorization Receipts for High-Risk Agent Actions. Internet-Draft draft-schrock-ep-authorization-receipts-13, September 2026 (-00: June 2026). https://datatracker.ietf.org/doc/draft-schrock-ep-authorization-receipts/

[10] I. Schrock. Binding Named-Human Authorization Evidence into Agent-Action Records. Internet-Draft draft-schrock-human-authorization-binding-00, July 2026. https://datatracker.ietf.org/doc/draft-schrock-human-authorization-binding/

[11] H. Kumaresan. counter-sign: An Open Protocol for Agent-to-Human Authorization. Internet-Draft draft-kumaresan-counter-sign-00, July 2026. https://datatracker.ietf.org/doc/draft-kumaresan-counter-sign/

[12] S. Mih. An Agent Action Capsule Profile for SCITT. Internet-Draft draft-mih-scitt-agent-action-capsule-05, September 2026. https://datatracker.ietf.org/doc/draft-mih-scitt-agent-action-capsule/

[13] R. Nelson. Delegation Receipt Protocol for AI Agent Authorization. Internet-Draft draft-nelson-agent-delegation-receipts-10, June 2026. https://datatracker.ietf.org/doc/draft-nelson-agent-delegation-receipts/

[14] P. Emirdag. AI Agent Execution Profile of SCITT. Internet-Draft draft-emirdag-scitt-ai-agent-execution-00, April 2026. https://datatracker.ietf.org/doc/draft-emirdag-scitt-ai-agent-execution/

[15] T. Toraman. A SCITT Profile for AI-Agent Action Receipts. Internet-Draft draft-noa-scitt-ai-agent-receipt-01, August 2026. https://datatracker.ietf.org/doc/draft-noa-scitt-ai-agent-receipt/

[16] T. Sato. The Human Escalation Mechanism (HEM) for Agentic AI Systems. Internet-Draft draft-sato-soos-hem-07, September 2026. https://datatracker.ietf.org/doc/draft-sato-soos-hem/

[17] T. Adebayo, O. Apalowo, F. Makanjuola. OMP Domain Profile: EU AI Act Article 12 Logging and Traceability Requirements for High-Risk AI System Operators. Internet-Draft draft-veridom-omp-euaia-00, October 2026. https://datatracker.ietf.org/doc/draft-veridom-omp-euaia/

[18] J. He, D. Yu. Sovereign Assurance Boundary: Certificate-Bound Admission for Agentic Infrastructure. arXiv:2606.11632, June 2026. https://arxiv.org/abs/2606.11632

[19] G. Racioppi. Mandato: Protocol-Level Enforcement of Digitally Signed Mandates on AI Agent Actions with Cryptographically Chained Audit Trails. arXiv:2608.14074, August 2026. https://arxiv.org/abs/2608.14074

[20] H. Birkholz, D. Thaler, M. Richardson, N. Smith, W. Pan. Remote ATtestation procedureS (RATS) Architecture. RFC 9334, January 2023. https://www.rfc-editor.org/rfc/rfc9334

[21] S. Torres-Arias, H. Afzali, T.K. Kuppusamy, R. Curtmola, J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. 28th USENIX Security Symposium, 2019.

[22] Z. Newman, J.S. Meyers, S. Torres-Arias. Sigstore: Software Signing for Everybody. ACM CCS, 2022.

[23] A. Mikulić. sar-cose: minimal reference implementation of the Sovereign Acceptance Runtime (SAR), version v1.1.0. Zenodo, 2026. doi:10.5281/zenodo.23270796. Source: https://github.com/rnfck88qng-a11y/sar-cose

## Revision Notes (v1.0 → v1.1)

- **Regulatory dates corrected.** v1.0 stated that the high-risk obligations (Arts 12, 14, 19, 26) apply from 2 August 2026. Regulation (EU) 2026/1744, in force since 27 July 2026, moved them to 2 December 2027 (Annex III) and 2 August 2028 (Annex I). Abstract, §1, §2.1, §5.2 and §5.7 are updated accordingly.
- **Prior art added.** New §2.6 discusses concurrent Internet-Drafts on agent authorization records, several of which predate v1.0. The gap statement (now §2.7) is narrowed from "no existing mechanism" to the specific composition SAR proposes. §7.2 and §8.2 now position SAR as complementary to those drafts.
- **Standards references updated.** The SCITT architecture is cited as RFC 9943. RFC 9864 (fully specified COSE algorithms) is noted.
- **Unsupported implementation claims replaced.** v1.0 tagged several subsystems and properties [Proven] and reported RQ1–RQ3 as "Supported" without an accompanying implementation. Those claims were withdrawn and are replaced by the results of the reference implementation below. §6 now reports implementation status and defines a conformance suite with mutation-based acceptance criteria. §4.7 and §4.9 are re-tagged [Specified].
- **Reference implementation added.** A COSE_Sign1/deterministic-CBOR reference implementation (sar-cose) with a 55-test conformance suite, differential and property-based tests, and 19-mutant mutation analysis now backs Section 6. Properties are re-tagged per Section 6.6; Receipts and transparency registration remain [Specified]. Testing found a gap in the original scenarios (ratification binding, test C13) and a concurrency defect that forked the hash chain (fixed).
- **Status line corrected.** v1.0's header read "Preprint submitted for SSRN / Zenodo", although no submission record was found; v1.1 is the first version published as a preprint (Zenodo, doi:10.5281/zenodo.23270905).
- **Author named.** The byline now names the author, matching the software record [23].
- **References section added.**